

LangChain Demo Document

Introduction

LangChain is a framework for developing applications powered by large language models (LLMs). It gives developers a standard set of building blocks, so instead of writing custom glue code for every model provider, prompt format, or data source, teams can compose a pipeline out of well-tested, interchangeable parts.

Core Building Blocks

Prompt Templates

Prompt templates let you parameterize the text sent to a model. Rather than hard-coding a prompt string, you define placeholders for variables such as user input, retrieved context, or chat history, and LangChain fills them in at runtime.

Chat Models

Chat models are wrappers around LLM providers such as OpenAI, Anthropic, Google Gemini, and Groq. They expose a consistent interface for sending messages and receiving responses, so switching providers rarely requires touching business logic.

Output Parsers

Output parsers take the raw text a model returns and convert it into a structured form, such as a Python dictionary, a list, or a validated Pydantic model. This makes it possible to reliably use LLM output in downstream code.

Document Loaders

Document loaders read data from a source, for example a local file, a web page, or a database, and convert it into a list of LangChain Document objects. Each Document has two parts: `page_content`, the raw text, and `metadata`, a dictionary describing where the text came from, such as its file path or page number.

Retrievers

Retrievers search a collection of documents for the pieces most relevant to a query. They commonly sit on top of a vector store, which holds numerical embeddings of document chunks so that semantically similar text can be found quickly.

Agents

Agents combine an LLM with a set of tools, such as a search API or a calculator, and let the model decide which tool to call and in what order, based on the task at hand.

Why Compose These Pieces Together

Each building block solves one narrow problem well. When chained together with LangChain Expression Language (LCEL), they form a pipeline: a prompt template formats a question, a chat model answers it, a retriever supplies supporting context, and an output parser shapes the final result into something an application can consume directly.

Example Use Case

A support chatbot might load past ticket transcripts with a document loader, index them with a retriever, and then use a prompt template and chat model to answer new customer questions by grounding its response in the retrieved transcripts. An output parser could then format the answer as JSON containing an answer field and a list of source citations.

Summary

LangChain does not replace the underlying LLM; it organizes the surrounding plumbing.

bing, prompts, memory, retrieval, parsing, and tool use, into reusable, swappable components so that applications can evolve without a full rewrite each time a requirement changes.